

Tarea N°1

Rutas Mínimas Berlín - Anaheim

Nombre
CI-5146
Profesores

Ariel Vidal
Redes de Transporte
Cristian Cortés
Ricardo De La Paz

Índice

1. Introducción	2
2. Metodología	2
a. Dijkstra	2
b. D'Esopo-Pape	3
3. Resultados	4
a. Berlín	4
b. Anaheim	5
4. Análisis	6
5. Conclusión	7

1. Introducción

Este informe contiene los procedimientos y resultados obtenidos en la tarea 1 del curso Redes de Transporte, consistente en encontrar las rutas mínimas para un set de pares Origen-Destino al interior de las redes de la ciudad de Anaheim, California y Berlín, Alemania.

El objetivo principal de esta tarea es la implementar computacionalmente métodos para resolver problemas de optimización en redes a partir casos reales. Para ello se implementan dos algoritmos, Label Setting (Dijkstra) y Label Correcting con manejo de colas tipo D'Esopo-Pape

La construcción de los arboles de rutas mínimas mediante los dos algoritmos implementados permite analizar el rendimiento de cada método en situaciones de diversa complejidad.

2. Metodología

Se utilizaron datos de las redes de transporte de dos ciudades, Berlín (Alemania) y Anaheim (Estados Unidos) proporcionadas por el equipo docente. Para representar ambas redes, se definen grafos dirigidos en forma de listas de adyacencia sobre un diccionario, donde cada clave corresponde a un nodo origen, almacenando tuplas que representan los arcos, se contiene al nodo destino y el costo del arco, definido como el tiempo de viaje a flujo libre. Por ejemplo, para el nodo 1 de Anaheim, se almacena lo siguiente:

$$1 : [(117, 1.09)]$$

La clave 1 corresponde al nodo origen , y se almacena una tupla compuesta por el nodo destino 117 y el costo asociado al arco equivalente a 1.09 unidades de tiempo. La representación mediante listas de adyacencia simplifica el acceso a los arcos salientes desde cada nodo, reduciendo la cantidad de información que debe recorrerse durante la ejecución de los algoritmos.

A. Dijkstra

Para la implementación del algoritmo label setting se utilizó una heap binaria. Para cada nodo se almacenó la menor distancia conocida desde el origen y el nodo predecesor asociado a dicha distancia. En cada iteración se extrae el nodo con menor distancia acumulada y se revisan los arcos salientes para buscar rutas a menor costo.

B. D'Esopo-Pape

La implementación del algoritmo D'Esopo-Pape se realiza a partir de una cola doble (deque). Para cada nodo se registra su estado (Nunca Visitado, En Cola, Procesado) para su identificación y manejo. Cuando una relajación mejora la distancia hasta un nodo previamente procesado, se reinserta a la cola con prioridad para propagar la actualización de distancias.

Una vez finalizada la ejecución de cada algoritmo, las rutas solicitadas son reconstruidas a partir del diccionario de predecesores que se genera tras aplicar cada algoritmo.

La siguiente tabla contiene las rutas solicitadas para cada ciudad.

Berlín		Anaheim	
Origen	Destino	Origen	Destino
322	751	127	380
	633		
	477		
	122		

Tabla 1: Pares OD de cada red a resolver.

3. Resultados

A. Berlín

Se aplicó el algoritmo de Dijkstra para resolver la red de Berlín, donde a partir del nodo origen 322 se construyó el árbol de rutas mínimas completo, obteniéndose las rutas y costos para los pares OD solicitados que se muestran en la tabla 2.

Origen	Destino	Ruta	Costo
322	751	322, 81, 374, 748, 972, 754, 2, 751	26.666
322	633	322, 380, 790, 793, 707, 685, 223, 7, 241, 243, 244, 962, 645, 670, 673, 648, 15, 660, 662, 661, 580, 579, 82, 571, 577, 559, 560, 637, 34, 633	129.333
322	477	322, 81, 374, 748, 972, 754, 2, 753, 5, 893, 801, 40, 796, 43, 721, 724, 44, 708, 681, 683, 682, 291, 135, 142, 263, 60, 272, 63, 268, 27, 132, 139, 50, 584, 67, 552, 553, 945, 950, 609, 68, 602, 69, 604, 601, 76, 603, 259, 839, 92, 611, 91, 215, 208, 436, 506, 507, 73, 508, 957, 74, 338, 347, 33, 476, 477	160.666
322	122	322, 81, 374, 748, 972, 754, 2, 753, 5, 893, 801, 40, 796, 43, 721, 724, 44, 706, 737, 736, 41, 701, 702, 825, 973, 826, 26, 768, 457, 59, 264, 62, 204, 219, 168, 157, 189, 210, 218, 54, 211, 113, 110, 85, 625, 121, 56, 122	121.999

Tabla 2: Rutas y costos obtenidos mediante Dijkstra para Berlín

De manera análoga, las rutas resultantes de aplicar el algoritmo D'Esopo-Pape en los pares OD solicitados junto con su respectivo costo se dan en la tabla a continuación:

Origen	Destino	Ruta	Costo
322	751	322, 81, 374, 748, 972, 754, 2, 751	26.666
322	633	322, 380, 790, 793, 707, 685, 223, 7, 241, 243, 244, 962, 645, 670, 673, 648, 15, 660, 662, 661, 580, 579, 82, 571, 577, 559, 560, 637, 34, 633	129.333
322	477	322, 81, 374, 748, 972, 754, 2, 753, 5, 893, 801, 40, 796, 43, 721, 724, 44, 708, 681, 683, 682, 291, 135, 142, 263, 60, 272, 63, 268, 27, 132, 139, 50, 584, 67, 552, 553, 945, 950, 609, 68, 602, 69, 604, 601, 76, 603, 259, 839, 92, 611, 91, 215, 208, 436, 506, 507, 73, 508, 957, 74, 338, 347, 33, 476, 477	160.666
322	122	322, 81, 374, 748, 972, 754, 2, 753, 5, 893, 801, 40, 796, 43, 721, 724, 44, 706, 737, 736, 41, 701, 702, 825, 973, 826, 26, 768, 457, 59, 264, 62, 204, 219, 168, 157, 189, 210, 218, 54, 211, 113, 110, 85, 625, 121, 56, 122	121.999

Tabla 3: Rutas y costos obtenidos mediante D'Esopo-Pape para Berlín

B. Anaheim

De forma análoga, se construyó el árbol de rutas mínimas con origen en el nodo 127, obteniéndose los resultados presentados a continuación.

El algoritmo de Dijkstra entrega las siguientes rutas y costos para los pares OD solicitados.

Origen	Destino	Ruta	Costo
127	380	127, 126, 125, 124, 123, 382, 381, 380	5.268
127	270	127, 350, 349, 156, 155, 154, 153, 152, 151, 150, 149, 148, 147, 146, 145, 144, 264, 265, 266, 24, 267, 268, 25, 269, 270	11.484

Tabla 4: Rutas y costos obtenidos mediante Dijkstra para Anaheim.

Para los mismos pares OD se encuentran las siguientes rutas y costos a través del algoritmo label correcting.

Origen	Destino	Ruta	Costo
127	380	127, 126, 125, 124, 123, 382, 381, 380	5.268
127	270	127, 350, 349, 156, 155, 154, 153, 152, 151, 150, 149, 148, 147, 146, 145, 144, 264, 265, 266, 24, 267, 268, 25, 269, 270	11.484

Tabla 5: Rutas y costos obtenidos mediante D'Esopo-pape para Anaheim.

Para cada modelo además se midió el tiempo computacional que toma correr el algoritmo, el número de iteraciones y número de nodos visitados. Lo anterior se resume en la siguiente tabla comparativa.

	Berlín		Anaheim	
	Dijkstra	D'Esopo	Dijkstra	D'Esopo
Tiempo Computacional	0.0031799	0.0023421	0.0006891	0.0008052
Iteraciones	957	1514	416	571
Nodos Visitados	957	957	416	416

Tabla 6: Métricas de rendimiento de cada algoritmo por ciudad.

4. Análisis

En la red de Berlín ambos algoritmos fueron capaces de construir el árbol completo de rutas mínimas con origen en el nodo 322 y se obtuvo rutas válidas en cada uno de los casos solicitados. Al comparar los resultados obtenidos, se observa que Dijkstra y D'Esopo-Pape generan exactamente la misma ruta (o secuencia de nodos) y costos para cada destino revisado; esto sugiere que ambas implementaciones funcionan de manera correcta y ambos algoritmos convergen a la misma solución óptima.

Si bien se obtuvo resultados idénticos con ambos algoritmos, se observa que Dijkstra necesitó menor cantidad de iteraciones para la construcción del árbol de rutas mínimas en comparación a D'Esopo-Pape, que necesitó procesar nodos de manera adicional por la corrección de etiquetas.

Para la red de Anaheim, ambos algoritmos presentaron un comportamiento similar al visto con la red de Berlín, ambos generaron con exactitud las mismas rutas mínimas para los dos pares OD solicitados. El obtener nuevamente los mismos resultados confirma que los algoritmos funcionan de manera correcta al encontrar la solución óptima.

Nuevamente, se tiene que el algoritmo de Dijkstra recurre a una menor cantidad de iteraciones que D'Esopo-Pape por el funcionamiento propio del algoritmo.

Un resultado interesante corresponde a la relación entre iteraciones y tiempo de ejecución observada en la red de Berlín. Si bien, el algoritmo de D'Esopo-Pape requirió de un 58.2% adicional de iteraciones que Dijkstra, esto no se tradujo en mayor tiempo computacional, por el contrario se encuentra que el tiempo utilizado por D'Esopo-Pape es un 26.3% menor. Esto sugiere que la cantidad de iteraciones realizadas no constituye una métrica lo suficientemente robusta para analizar el desempeño por sí sola.

Lo anterior se revierte en el caso de Anaheim, para esta red se tiene que el algoritmo de D'Esopo-Pape requiere de mayores recursos que Dijkstra, realizándose un 37.3% adicional de iteraciones. El tiempo que el algoritmo Label Correcting también es mayor por un 16.8%.

Un aspecto relevante es que ambos algoritmos visitaron exactamente la misma cantidad de nodos en ambas redes. Esto indica que las diferencias de rendimiento observadas no solo dependen del tamaño de la red, sino que del propio funcionamiento del algoritmo en conjunto con la complejidad y estructura de la red.

5. Conclusión

La implementación de ambos algoritmos para la resolución de las rutas mínimas entregó resultados que muestran la capacidad de construir correctamente los árboles de rutas mínimas desde los orígenes definidos y encontrar rutas válidas (y óptimas) para los pares OD solicitados por enunciado.

La comparación realizada verificó que ambos algoritmos generan las mismas rutas, por lo que las diferencias encontradas surgen por la manera en que cada uno de los algoritmos explora la red y afectan de manera exclusiva al rendimiento y recursos computacionales consumidos.

Los resultados obtenidos muestran también que el tamaño y complejidad de la red influye de manera directa en el esfuerzo computacional requerido, en particular, la red de Berlín presenta tiempos de ejecución superiores a la red de Anaheim por la cantidad de nodos significativamente mayor.

Las tendencias observadas en las métricas de cada algoritmo muestran que el desempeño relativo de cada algoritmo depende de las características de la red analizada. Pese a esto, ambos métodos cumplen de manera satisfactoria su objetivo sin comprometer la calidad de las soluciones encontradas.

Finalmente cabe destacar que las redes utilizadas además poseen arcos que se encuentran restringidos por su capacidad, sin embargo las rutas encontradas en esta tarea se calculan bajo condiciones a flujo libre, por lo la ruta mínima final puede variar si la asignación de flujos introduce congestión al sistema.

- **Link a Github**

<https://github.com/arielvidalvidal/Tarea-1.git>